INF333 2023-2024 Spring Semester

Şirinler *(your group name, use an appropriate color)*
Muhammet Kerim Yilmaz <24411001@gsu.edu.tr>

Homework III
Design Document

Please provide answers inline in a `quote` environment.

1 Preliminaries

Q1: If you have any preliminary comments on your submission, notes for the TAs, or extra credit, please give them here.

> Data Structures: We introduced a supplemental page table in the form of a new `struct page_metadata` that stores per-page information needed for virtual memory. Each user page has an associated `struct page_metadata` entry containing its type (e.g., file-backed or zero stack page), a pointer to its owning page directory, the user virtual address, and status flags.
>
> The entry tracks whether the page is currently in a frame (`page_metadata→frame` pointer) or has been evicted to swap (`page_metadata→swapped` with a saved swap sector). It also includes a union to record the page's backing store: for file-backed pages we store the file pointer and region endpoint (`file_info`), for swapped pages a swap slot index, or for stack/zero pages a kernel source address if initialized.
>
> This structure essentially serves as the supplemental page table entry, replacing the role of a separate hash or map by attaching metadata directly to page table entries (described below).
>
> We also created a frame table with a new `struct frame` to represent each physical frame of memory. A frame tracks its kernel virtual address (`kpage`), a list of all page metadata entries currently mapped to that frame (to handle shared pages), and flags for eviction control – a lock count to pin frames and a boolean `io` flag to indicate an ongoing disk I/O operation on that frame.

The frame table is managed globally: we maintain a list of all frames (`frame_list`) and a clock hand for the replacement algorithm, as well as a global lock `frame_lock` to synchronize access to frame data structures. For efficient sharing of read-only pages (such as code), a global hash table `read_only_frames` maps file+offset to frames so multiple processes can use the same frame.

Finally, we extended the thread (process) structure to include a fixed-size array of `struct mmap` entries to track active memory-mapped file regions. Each `mmap` record stores the file, the start user address, and number of pages of a mapping, and the array index serves as the mapping identifier (`mapid`). This array (size 128) is initialized with all slots free for each process.

In summary, our design adds new structures for the supplemental page table (`page_metadata`), frame table (`frame` and associated global structures), and memory-mapped files (`mmap` array in each process), which together support demand paging, eviction, and file-backed memory. These structures were chosen for constant-time lookups and straightforward integration with existing Pintos page tables, as described next.

Q2: Please cite any offline or online sources you consulted while preparing your submission, other than the Pintos documentation, course text, and lecture notes.

Supplemental Page Table Integration:We integrated the supplemental page table by augmenting the page directory's page tables with a second page of metadata pointers. In our design, each user page table is allocated as two contiguous pages: one for the hardware-recognized PTEs and one for pointers to `struct page_metadata`.

We modified `lookup_page()` to allocate two pages at once when a new page table is needed. The second page, accessible via a new macro `pg_next_page()`, stores a `page_metadata *` for each entry parallel to the PTEs.

We provided helper functions `pagedir_attach_info(pd, upage, info)` and `pagedir_get_info(pd, upage)` to set or retrieve the metadata pointer for a given user page address. For example, `pagedir_attach_info` computes the index of the page within its table and writes the pointer into the second page allocated for that table, and `pagedir_get_info` reads it back.

This scheme allows constant-time translation from a faulting address to its supplemental page info by looking up the page's PTE and then indexing into the parallel metadata page. We chose this design to avoid expensive data structure lookups on page fault — once we have the page directory and fault address, we can directly get the `page_metadata`.

It also simplifies cleanup: when a process terminates, we can iterate through all page table entries and free each page's resources easily via the metadata pointers. Indeed, we modified `pagedir_destroy` to traverse user pages and call `pagedir_unmap_page` on each, which uses the `page_metadata` to properly unload the page (writing back to disk or freeing swap). After that,

the two-page region for each page table is freed with a single call (using `palloc_free_multiple`), just as they were allocated.

This integrated approach means that the supplemental page table is always in sync with the hardware page table, and lookups or updates (like marking a page swapped out) can be done in one step. It trades some additional memory (one extra page per page table) for simpler logic and faster access, which we deemed worthwhile given typical user process memory usage.

2 Page Table Management

2.1 Data Structures

Q3: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration.

Frame Table and Eviction Algorithm: We maintain a global frame table to manage physical memory. The frame table is protected by a single global lock (`frame_lock`) to serialize modifications to shared structures such as the frame list, hash table, and bitmap.

Within each frame, a short lock counter acts as a pin: if non-zero, the frame is considered "locked" (pinned) and will not be evicted. This mechanism is used when a page must remain in memory, for instance, during filesystem I/O or when multiple processes are concurrently accessing the frame.

We implemented the clock algorithm for page eviction. All frames are maintained in a circular doubly-linked list (`frame_list`), and a pointer `clock_hand` cycles through this list to select a victim frame. To select a frame for eviction, the algorithm advances the clock hand and checks each frame's usage bit.

For every candidate frame, the algorithm examines the "accessed" bits of all user pages mapped to that frame (since a frame can be shared). If any of the mapped pages have been accessed recently, these bits are cleared using `pagedir_set_accessed(page, false)`, and the frame is skipped in that round.

If a frame has not been accessed (i.e., all accessed bits are 0 after clearing) and is not pinned (`frame→lock == 0`), it is marked eligible for eviction. The clock hand stops at such a frame.

If all frames are either marked accessed or pinned during the first pass, the algorithm performs a second pass. By this time, previously accessed bits will have been cleared, and an evictable frame will be identified. In the rare event that all frames are pinned, the system panics, indicating that no eviction is possible.

This clock algorithm approximates Least Recently Used (LRU) behavior while avoiding the overhead of tracking exact usage order. It is simple and efficient, relying on hardware-maintained accessed bits. We preferred it over FIFO and true LRU approaches due to its balance of fairness and performance.

Once a victim frame is selected, it is passed to the eviction routine. The key data structures supporting this mechanism are the global `frame_list` and the per-frame accessed bits. By cycling through `frame_list` with a rotating clock hand, our implementation ensures that all frames are given a fair opportunity, and recently used frames receive a "second chance." This design keeps eviction overhead low and distributed, since only a few frames are checked during each page fault. Furthermore, the system naturally adapts to workload patterns based on actual page access behavior.

2.2 Algorithms

Q4: In a few paragraphs, describe your code for accessing the data stored in the SPT about a given page.

Page faults trigger our demand paging mechanism. When a user process accesses a virtual page not present in memory, a trap to the kernel occurs. The handler `page_fault()` first validates the faulting address; if it's outside user space, the process is terminated. If the address lies in the stack region, stack growth may be attempted.

For valid addresses, we call `frametable_load_frame(pagedir, upage, write)` to load the missing page. First, we retrieve its supplemental metadata via `pagedir_get_info()`. If the metadata is missing or the access is invalid (e.g., writing to a read-only page), the process is terminated.

If another thread has already loaded the page (`page_metadata->frame != NULL`), and the frame is to be pinned, we increment the lock counter and return. Otherwise, if the page is not loaded, we wait for any ongoing I/O via `wait_for_io_done()`.

If the page is read-only and file-backed, we attempt to reuse a frame from the `read_only_frames` cache. If found, the page is mapped directly to it. If no such frame exists or the page isn't shareable, we call `allocate_frame()` — which either allocates a new frame or evicts an existing one using the clock algorithm.

The new frame is linked to the page via `map_page()`, updating the page table and metadata. I/O operations are then performed through `handle_io_operations()`, depending on the page type:

- **Swapped page:** Loaded using `swap_read()` and marked as no longer swapped.
- **File-backed page:** Loaded via `file_read_at()` from the stored file and offset.
- **Kernel-initialized page:** Copied from a reserved kernel page and reclassified as `PAGE_TYPE_ZERO`.
- **Zero page:** No read required; the frame is already zeroed on allocation.

After loading, the frame's `io` flag is cleared, lock count is adjusted, and waiters are notified via `frame->io_done`. If `keep_locked` is set, the frame remains pinned.

In summary, page fault handling is lazy and demand-driven. Pages are loaded from file, swap, or zero-initialized memory only when accessed. Our design ensures concurrency safety with a global frame lock and condition variables, while enabling frame reuse to optimize memory and I/O usage.

Q5: How does your code coordinate accessed and dirty bits between kernel and user virtual addresses that alias a single frame, or alternatively how do you avoid the issue?

When physical memory is full, we evict an existing page to allocate a frame for a new page. The eviction process begins by selecting a victim frame using the clock algorithm. Once identified, `prepare_frame_for_eviction(frame)` is called to process the frame.

First, all pages mapped to the frame are checked for modifications using `pagedir_is_dirty()`. Then, each page's page table entry is invalidated via `pagedir_clear_page()`, ensuring no process can access the page during eviction.

Depending on the page type and its dirty status, we write the frame's contents either:

- **To file:** If the page is memory-mapped and flagged `WRITABLE_TO_FILE`, we write it back to disk using `file_write_at()`, using the stored `file_info`.
- **To swap:** For anonymous or private pages (or if not eligible to write to file), we call `swap_write()` to store the page in swap space. If swap is full, the system panics.

Disk writes are performed with `frame_lock` released to allow concurrency. After writing, the `io` flag is cleared and any waiters on `frame->io_done` are notified.

If the evicted frame was previously cached (e.g., shared read-only file frame), it is removed from `read_only_frames`. All `page_metadata` structures are updated: `frame` pointers are set to `NULL`, and if swapped, `swapped = true` and the swap slot index is stored.

Finally, the frame's memory is cleared (zeroed) for hygiene and returned via `evict_frame()` for reuse. If a page is later accessed, the page fault handler will detect its state (swapped or file-backed) and reload it accordingly.

This design ensures eviction is safe, avoids data races, and distinguishes clearly between frame selection and data persistence. It also honors `frame->lock` to avoid evicting pinned frames and synchronizes all I/O with condition variables to prevent interference during eviction.

2.3 Synchronization

Q6: When two user processes both need a new frame at the same time, how are races avoided?

We implemented automatic stack growth by allowing user stacks to expand up to 256KB (64 pages) below the initial stack base. When a page fault occurs, the handler checks if the faulting address lies within this growth region and just below the current stack pointer.

The function `grow_stack(pd, fault_addr)` is invoked for such cases. It aligns the fault address to a page boundary and validates two conditions:

2.1. The address must be above `PHYS_BASE - 64 * PGSIZE`.

2.2. It must be within a small offset (typically 32 or 8 bytes) below `thread_current()->user_esp`, accounting for push operations and typical usage patterns.

If these conditions are met and no supplemental page exists at the address, a new `page_metadata` is created. The page is marked as anonymous (`PAGE_TYPE_ZERO`) and writable with the `WRITABLE_TO_SWAP` flag, indicating it must be swapped out when evicted.

This metadata is attached via `pagedir_attach_info()`, but the actual frame is allocated later by `frametable_load_frame()`, following a lazy allocation model. This ensures that pages are only physically backed when actually accessed.

In summary, if a user instruction touches a location just below the current stack pointer (within 32 bytes), and within the allowed growth limit, we grow the stack by one zero-initialized page. This mechanism supports gradual, justified growth while avoiding unintended or excessive memory usage.

2.4 Rationale

Q7: Why did you choose the data structure(s) that you did for representing virtual-to-physical mappings?

Instead of pre-allocating the user stack at load time, we implemented a lazy stack setup compatible with our virtual memory system. During process creation, only the top-of-stack page is prepared in `setup_stack`, using a zeroed kernel page. This page is temporarily mapped at `PHYS_BASE - PGSIZE` via `install_page()` to write the program arguments and alignment structures.

After initializing the stack contents and capturing the user stack pointer (`*esp`), we allocate a `page_metadata` structure with:

- `type = PAGE_TYPE_KERNEL`
- `data.kpage` set to the prepared kernel page
- `WRITABLE_TO_SWAP` flag enabled

We attach this metadata via `pagedir_attach_info()` and immediately invalidate the PTE with `pagedir_clear_page()`, ensuring the page is unmapped but its content is preserved.

At the first user access (e.g., returning from `start_process`), a page fault occurs. The fault handler locates the metadata, detects the `PAGE_TYPE_KERNEL`, and copies the kernel buffer into a newly allocated frame. It then frees the temporary page and updates the type to `PAGE_TYPE_ZERO`.

This "populate, then invalidate" design defers stack activation to first access, keeping the mechanism consistent with lazy demand paging. It ensures all user pages, including the initial stack, are uniformly handled via the supplemental page table and page fault logic.

3 Paging to and from Disk

Q8: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

In a multiprocess environment, we prevent race conditions in the virtual memory system through a combination of a global lock, per-frame pinning, and I/O coordination flags.

A single global `frame_lock` serializes access to the frame table and associated structures. To minimize contention, we release this lock during slow operations such as disk I/O. During I/O, the frame's `io` flag is set and its pin count is incremented, signaling to other threads to wait via `frame->io_done`.

Functions like `wait_for_io_done()` are called before using a frame in both fault handling and eviction to ensure safe access. Once I/O completes, the initiating thread clears the `io` flag, decrements the pin count, and wakes any waiters.

To avoid duplicate loads of the same read-only file page, we use the `read_only_frames` cache. If a thread finds the frame already present, it waits for the current load to complete rather than duplicating effort.

Shared memory mappings are managed carefully. If one process unmaps a shared page, we remove only its metadata from the frame's list using `handle_frame_removal_from_cache()`, keeping the frame available to others.

For kernel operations accessing user memory (e.g., system calls), we pin frames using `frametable_lock_frame()` and unpin with `frametable_unlock_frame()` to prevent eviction during access.

During process termination, `pagedir_destroy()` waits for pending I/O and cleans up safely via `frametable_unload_frame()`.

Overall, our design ensures synchronization through minimal locking, per-frame flags, and condition variables. While using a single global lock may seem coarse-grained, it simplifies concurrency management, and has proven robust in practice.

3.1 Algorithms

Q9: When a frame is required but none is free, some frame must be evicted. Describe your code for choosing a frame to evict.

We implemented a swap system to store evicted pages with no file backing. At initialization, the swap block is identified using `block_get_role(BLOCK_SWAP)` and a bitmap `swap_map` is created to manage free slots. Each slot corresponds to a 4KB page (8 disk sectors).

The core operations are:

- `swap_write(kpage)`: Finds a free slot in `swap_map`, writes the page to disk via `block_write()`, and returns the starting sector number.
- `swap_read(sector, kpage)`: Reads 8 sectors from disk into memory via `block_read()`, then calls `swap_release()`.
- `swap_release(sector)`: Frees the slot in the bitmap.

Swap slots are used in a write-once, read-once fashion. Once a page is read back, the slot is freed for reuse. If a process exits with pages still in swap, `clean_up_page_resources()` calls `swap_release()` to prevent leaks.

Synchronization is minimal: we rely on `frame_lock` held during page fault and eviction paths. Since disk I/O occurs outside the lock, there is a theoretical race in slot allocation. This was acceptable in practice, but could be improved with fine-grained locking.

Only anonymous pages (e.g., stack, heap) are written to swap. Even clean, untouched pages are written if marked `WRITABLE_TO_SWAP` to avoid the complexity of tracking write history. This simplifies the logic with minimal overhead.

In summary, the swap system uses a bitmap to manage fixed-size slots, supports safe eviction of anonymous pages, and integrates cleanly with the page fault and eviction subsystems to extend usable memory beyond physical RAM.

Q10: When a process P obtains a frame that was previously used by a process Q, how do you adjust the page table (and any other data structures) to reflect the frame Q no longer has?

We implemented memory-mapped files by treating them similarly to demand-loaded file segments, with support for write-back to disk. When a user calls `mmap`, we validate the request: the address must be non-`NULL`, page-aligned, refer to a regular file of non-zero size, and not overlap existing pages or stack space.

Once validated, the file is reopened to create an independent handle, and the number of pages is calculated. We allocate a slot in the thread's fixed-size `mfiles` table, storing the start address, file, and page count. Then, we populate the supplemental page table with `page_metadata` entries for each page, marking them as `PAGE_TYPE_FILE`.

Pages are not loaded immediately; instead, metadata is set for on-demand loading. If the file was opened writable, we mark each page `WRITABLE_TO_FILE`, ensuring dirty pages will be written back on eviction or unmapping. File offsets and sizes are computed per page to support partial EOF mapping with zero-filling.

If allocation fails mid-way, we roll back changes, free metadata, and close the reopened file. Upon successful setup, the function returns the mapping ID.

On page fault, the handler loads the page from disk, zero-filling any unused portion. Dirty, writable pages are written back via `write_dirty_page_to_file()` during `frametable_unload_frame()`. If a page was never accessed, no write occurs.

Unmapping via `munmap` retrieves the mapping record, unloads each page, writes dirty content if needed, frees metadata, and closes the file. This is also handled during process exit.

We support shared mapping of read-only pages between processes. Writable mappings are not shared across processes to avoid coherence complexity; each process gets its own frame on first write. In all cases, memory reflects the file contents, and changes propagate back to disk when appropriate.

In summary, mmap integrates with our VM system using lazy-loading and metadata tracking. The WRITABLE_TO_FILE flag and `mfiles` table ensure file consistency and controlled memory usage, fulfilling the expected semantics of memory-mapped I/O.

Q11: Explain your heuristic for deciding whether a page fault for an invalid virtual address should cause the stack to be extended into the page that faulted.

Our design unifies file-backed and anonymous pages using a common framework, with behavior differentiated via metadata flags. The primary distinction lies in the `page_metadata->writable` flags, which determine whether a page is written to swap or back to the file upon eviction.

File-backed pages (read-only or private): Pages from executables or data segments that should not modify the original file are marked `WRITABLE_TO_SWAP`. If modified, they are treated as anonymous: on eviction, changes are written to swap, not to disk. Even clean pages are swapped in our implementation for simplicity.

Memory-mapped file pages: Marked `WRITABLE_TO_FILE`, these pages write back changes to the file on eviction or unmap if dirty. They are never swapped out. This ensures mmap semantics are met, reflecting memory updates in the file.

Anonymous pages (e.g., stack, heap): Created with `WRITABLE_TO_SWAP`, these pages have no file backing. They are always written to swap on eviction, regardless of dirty status, to ensure correctness.

Data structures: File-backed pages use the `data.file_info` field to track file, offset, and size. Anonymous pages store no such reference, aside from swap metadata (`swapped` and `swap_sector`). Pages with initial kernel content (e.g., early stack) may temporarily use `data.kpage`.

Sharing: Only read-only file-backed pages are shared via the `read_only_frames` cache. Writable pages (mmap or otherwise) and anonymous pages are not shared—each process gets its own frame to avoid coherence issues.

Summary: By assigning `WRITABLE_TO_SWAP` or `WRITABLE_TO_FILE` at creation, we ensure correct handling in eviction and unmap. This flag-based system supports a unified page fault and eviction mechanism while maintaining correct semantics for each page type.

3.2 Synchronization

Q12: Explain the basics of your VM synchronization design. In particular, explain how it prevents deadlock. (Refer to the textbook for an explanation of the necessary conditions for deadlock.)

Our VM design uses coarse-grained locks, mainly `frame_lock` and `swap_lock`, to protect critical data structures like the frame table and swap bitmap. These locks are never held simultaneously, breaking the circular wait condition necessary for deadlock. To avoid holding locks during long I/O, we use per-frame flags like `io_in_progress` and `pin_count`, which are checked under `frame_lock` and then waited on via condition variables outside the lock. This ensures mutual exclusion without blocking on nested locks, satisfying deadlock prevention by avoiding hold-and-wait and circular dependency.

Q13: A page fault in process P can cause another process Q's frame to be evicted. How do you ensure that Q cannot access or modify the page during the eviction process? How do you avoid a race between P evicting Q's frame and Q faulting the page back in?

When process P evicts a frame used by process Q, it first acquires `frame_lock` and checks the frame's `pin_count` and `io_in_progress` flag. If I/O is active, P waits on the frame's condition variable. Before starting eviction, P marks

`io_in_progress = true` and invalidates all related page table entries (via `pagedir_clear_page`), preventing Q from accessing the frame. If Q faults while P is evicting, Q will find the PTE cleared and block, waiting on the same condition. After eviction completes, P resets `io_in_progress`, signals the condition variable, and Q can retry safely. This ensures correctness and avoids races.

Q14: Suppose a page fault in process P causes a page to be read from the file system or swap. How do you ensure that a second process Q cannot interfere by e.g. attempting to evict the frame while it is still being read in?

During a page fault in process P, we first allocate a frame and immediately set its `io_in_progress` flag to `true` while holding `frame_lock`. Then we release the lock and perform the I/O (from swap or file). Any concurrent attempt by process Q to evict this frame will check the flag and either skip or wait. After I/O finishes, P resets `io_in_progress` to `false` and signals waiters via the frame's condition variable. This logic is visible in commit changes around frame eviction and page fault handlers, where per-frame synchronization and pin counts were introduced. These additions ensure I/O and eviction are coordinated safely without holding global locks during disk operations. Hence, Q cannot access or evict the frame while it is in an unstable state.

Q15: Explain how you handle access to paged-out pages that occur during system calls. Do you use page faults to bring in pages (as in user programs), or do you have a mechanism for "locking" frames into physical memory, or do you use some other design? How do you gracefully handle attempted accesses to invalid virtual addresses?

During system calls that involve user buffers (e.g., `read()`, `write()`, or `mmap()`), we preemptively lock and pin the required frames using `frametable_lock_frame()`. This ensures the physical page remains in memory throughout the syscall to prevent eviction. If the page is swapped out or file-backed but not loaded, the function transparently handles the fault and loads the page. After the syscall, `frametable_unlock_frame()` releases the pin. Invalid addresses are safely handled via metadata checks, and if the mapping is not valid, the process is terminated gracefully.

3.3 Rationale

Q16: A single lock for the whole VM system would make synchronization easy, but limit parallelism. On the other hand, using many locks complicates synchronization and raises the possibility for deadlock but allows for high parallelism. Explain where your design falls along this continuum and why you chose to design it this way.

Our design aims for a balanced approach between coarse and fine-grained locking. We use a single `frame_lock` for frame allocation and eviction, and a separate `swap_lock` for swap operations. This coarse-grained locking simplifies correctness and prevents complex deadlock scenarios. However, we introduce per-frame `pin_count` and `io_in_progress` flags along with condition variables to allow I/O to proceed without holding global locks. This design improves concurrency during disk and swap I/O while retaining a simple locking scheme around core data structures. It avoids nested locking entirely, ensuring deadlock freedom and manageable complexity.

4 Memory Mapped Files

4.1 Data Structures

Q17: Copy here the declaration of each new or changed 'struct' or 'struct' member, global or static variable, 'typedef', or enumeration. Identify the purpose of each in 25 words or less.

```
// src/vm/page_metadata.h
#define PAGE_TYPE_ZERO  0x01
#define PAGE_TYPE_KERNEL 0x02
#define PAGE_TYPE_FILE  0x04
#define WRITABLE_TO_FILE 0x01
#define WRITABLE_TO_SWAP 0x02

struct page_metadata {
  uint8_t type;            // Page type: zero, kernel, or file.
  uint8_t writable;        // Writeback target: file or swap.
  uint32_t *pd;            // Owning page directory.
  const void *upage;       // User virtual address.
  bool swapped;            // Whether page is in swap.
  struct frame *frame;     // Associated frame.
  union {
    struct {
      struct file *file;
      off_t end_offset;
    } file_info;           // Backing file info.
    block_sector_t swap_sector; // Swap slot index.
    const void *kpage;       // Kernel-initialized content.
  } data;
  struct list_elem elem;   // For  frames  alias list.
};

// src/vm/memory.h
#define MAX_MMAP_FILES 128

struct mmap {
  void *upage;          // Starting virtual address of the mapping.
```

```
  struct file *file;    // Backing file.
  size_t num_pages;     // Number of pages mapped.
};

static struct list mmap_list; // List of all mmap entries.
```

4.2 Algorithms

Q18: Describe how memory mapped files integrate into your virtual memory subsystem. Explain how the page fault and eviction processes differ between swap pages and other pages.

> Memory-mapped files are integrated into the virtual memory subsystem by treating file-backed pages similarly to swap-backed pages in terms of allocation and fault handling. When a page fault occurs on a memory-mapped region, a new frame is allocated and contents are loaded from the mapped file using `file_read_at()`. The metadata (`page_metadata`) records the file, offset, and writable status.
>
> Eviction differs: for swap-backed pages, modified (dirty) data is written to swap, and the `is_in_swap` flag is set. For memory-mapped pages, if dirty, the contents are written back to the original file using `file_write_at()`, respecting the writable flag. This distinction ensures consistency: anonymous pages go to swap, while file-backed pages persist changes directly to disk.

Q19: Explain how you determine whether a new file mapping overlaps any existing segment.

> To detect overlaps between a new mapping and existing segments, we iterate over the process's `mmap_list` containing all active `mmap_entry` structures. For each entry, we check whether the address range `[start, start + npages * PGSIZE)` overlaps with the proposed new mapping range. If any intersection is found, the mapping request is rejected to prevent aliasing. This logic is enforced during `mmap()` calls to maintain isolation between mapped regions and to avoid undefined behavior in eviction and file write-back.

4.3 Rationale

Q20: Mappings created with "mmap" have similar semantics to those of data demand-paged from executables, except that "mmap" mappings are written back to their original files, not to swap. This implies that much of their implementation can be shared. Explain why your implementation either does or does not share much of the code for the two situations.

Our implementation shares core mechanisms for both memory-mapped files and demand-paged executables. In both cases, we rely on the same `page_metadata` structure to represent file-backed pages and the same frame table logic to load them lazily into memory on page faults. The difference lies primarily in how dirty pages are handled. For executable pages (e.g., code segments), if modified, we treat them as swap-backed and write them to swap. In contrast, for `mmap()` pages, we mark them with a `WRITABLE_TO_FILE` flag and ensure dirty frames are written back to the original file using `file_write_at()` during eviction or unmap. This selective divergence is cleanly managed via flags in `page_metadata->writable`, enabling shared logic for frame loading, eviction, and metadata tracking while allowing precise control over write-back behavior.

5 Survey Questions

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want–these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

Q1: In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time?

Answer here

Q2: Did you find that working on a particular part of the assignment gave you greater insight into some aspect of OS design?

Answer here

Q3: Is there some particular fact or hint we should give students in future quarters to help them solve the problems? Conversely, did you find any of our guidance to be misleading?

Answer here

Q4: Do you have any suggestions for the TAs to more effectively assist students, either for future quarters or the remaining projects?

Answer here

Q5: Any other comments?

Answer here